

Competitive Simon Says

Aaron Chau

Parker Macomber

Helena Gonzales

Jaehoon Ryu

Armani Vang

7 May, 2025

University of Colorado Denver

CSCI 3515

Final Project Report

Introduction

Our final project is based on the game “Simon”. Simon is a small handheld game with 4 buttons that correspond to 4 colored lights, red, blue, yellow, and green. The game begins with one light flashing, after the player successfully presses the matching the color sequence of lights and so forth, the game will progressively get longer and longer until a player fails to mimic the sequence. This is a very simple idea to implement on an arduino using buttons, LED pins, and passive buzzers. To further expand on this idea we turned Simon into a two player game.

The competitive nature of our implementation comes from the back and forth communication of two arduino. The boards have identical game layouts, including buttons and lights. Each board does have its own sequence to follow and each player will have a turn. Once the turn is completed the other player will get a chance to do a sequence with the same number of blinking lights. The game continues until a player cannot repeat the sequence and thus the other player wins.

Design, implementation, and challenges will be further explored later in the report, but here is a brief summary. Smaller design choices we made all correlate to an easier user experience, so sound cues are implemented throughout the game. A sound is made to signify that the player currently in control has switched. A sound cue also plays to notify that the game is completed. Sound was opted for over an LCD display due to sound being more beneficial in a multiplayer setting. An LCD display also was not used due to one of the challenges we encountered, not enough space. The implementation of a buzzer took up less space than a working LCD display. More challenges and creative thinking arose during the arduino to arduino communication and this will be elaborated on during the final section of the report.

Design

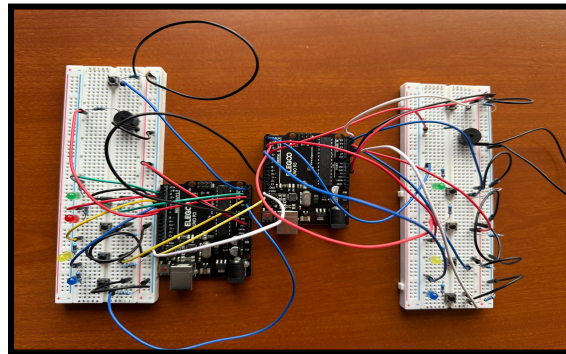
The design of our final competitive Simon Says includes three major factors that made the final product successful. Those three parts consisted of button actions/LEDs, buzzer implementation for the sound, and most importantly using I2C communication to send and receive information between both devices.

When building the original simon game, we needed the essential parts of hooking up buttons to LEDs. We were able to do this by taking four different colored LEDs (Blue, Yellow, Red, Green) and connecting them to the arduino board with 220 Ω resistors. Then we were able to add a button next to its corresponding colored LED with 10k Ω resistors, which is also hooked up to the arduino board. When the code is uploaded to the arduino board the code will generate a sequence that tells what LED to light up and the corresponding button will be used by the player to match the sequence that is given. When the user presses the button, the LED will light up with the corresponding button. When the sequence is complete the LEDs will flash. It is also noted that there is a fifth button that is used to start the game also uses a 10K Ω resistor like the other buttons.

When building the Connection between two simon games, we used essentially the same set up used in the I2C Communication Lab (Lab 13). This is a simple part of the design because it only needs the connection of three wires between both arduino boards. We needed to be able to connect the SCL and SDA ports of both arduino boards. These two ports are ideal for sending information back and forth. It is also important that we make a GND connection, such that the boards can be grounded. When the code is uploaded to the arduino boards the signals being sent between each board are to alert if the other board has completed its sequence or has failed the sequence, telling the other board to keep on going or to stop.

Finally, for the final major component of our build, we needed a way to make sound on different indications that occur during the game. To do this we used a passive buzzer from the arduino kit. To set up this buzzer, we used inspiration from the buzzer implementation lab (Lab 8). The buzzer itself used two wires, one taking in power from the board at pin 7 and one that grounded the passive buzzer. The buzzer also uses a 220 Ω resistor, as this resistor used the loudest and clearest sound for our game. The idea for this buzzer sound was to indicate when a pass between two players occurs and when a player gets the sequence correct. The buzzer is also used at the end of a game for when a player loses the game. In this case we used a Mario theme song to play when this happens. Below is a photo of our design, that being wiring and placement of devices, where everything above is shown on the real life competitive Simon Says game.

Final Design:



Implementation

This project contains two different parts with the I2C communication. An Initiator and a responder. In the I2C communication, the initiator sends a byte (1) to the responder at an address (8) when a round is completed. The initiator then actively checks to see if the responder is finished with their turn. The setup function starts the I2C communication for the initiator and initializes the pins used on the Arduino board. The loop function handles the game flow. If myTurn is false, the responder will wait. If the level equals 1, a random sequence is generated. If

the start button is pressed, the show sequence and get sequence functions are called. The show_sequence() function displays the current sequence and plays a buzz to indicate a player's turn is starting. The get_sequence() function collects and validates the user's input. Depending on if the input is correct or incorrect, the right_sequence() or wrong_sequence() will be called. The right_sequence() function will play a success sound indicating that the game will continue, and advances the level with an increasing speed. The wrong_sequence() function will play a game over sound and directly end the game for both users as it flashes the LEDs on the board.

On the responder side, the code mainly reacts to what the initiator sends over I2C. When the initiator sends a signal, the receiveEvent() function gets triggered and flips a flag to start the round. If it's the beginning of a new game, the responder creates a random LED sequence using generate_sequence(), then flashes that pattern with show_sequence(). After the pattern is shown, the responder waits for the player to press the buttons in the correct order. In get_sequence(), each input is checked step by step. If the input matches the full pattern, right_sequence() gives visual and audio feedback, moves to the next level, and marks the round as finished. If there's a mistake, wrong_sequence() resets the level and plays a failure sound using the buzzer. While all of that is happening, the initiator can call requestEvent() to check whether the responder is done. Overall, the responder handles the actual game logic like pattern display and input checking. The initiator takes care of managing the turns and starting the next round.

Challenges

Some challenges we faced caused us to reconsider a few elements of our main project. Previously stated, we originally were going to have a LCD display, showing who's turn it was, but we decided to remove that component and keep our buzzer. The challenges that came with

this were only a few, but these issues gave our team a better outcome. Firstly, trying to implement our separate codes to notify our users was our trickiest challenge we solved. We had to make sure that a trigger would occur to the other arduino to notify the other player to play by making a sound. Making a new function in both our codes had to be implemented so the correct pitches could be used and make a winning and losing sound. As said before, we used a previous lab to assist us on this challenge and were able to figure out how to make the correct delays and make the correct pitches so our players could play.

A different issue, that doesn't involve sound, was our lights. At some point when we tested our game, the lights on our board would not turn on or were not able to be played during a game. We had checked to see if our code was the main issue, but it was our board that had complications with the button we used. The correct current was not sent correctly to the positive side of our LED, triggering it to turn on, and with a switch of our button our lights managed to keep light after a long practice round. This happened to more than one LED, but thankfully our changing buttons allowed us to make our game playable.

All in all, our challenges dealt with our lighting and sound, but we have resolved those issues and made our final project. Even though we initially had a LCD display in our proposal, because of space, we decided that removing that display and keeping our buzzer would make our Simon Says game more fun. Since this game can be played over 100 levels for a single player, this game can be played competitively with two players.